

PRIMERA ACTIVIDAD: ¿Cómo abstraer algo del mundo real a la programación?

Caso de Estudio: Sistema de Scooters Eléctricos

Objetivo: Comprender la estructura de un sistema orientado a objetos identificando clases, atributos y métodos, diferenciando el concepto abstracto (Clase) de su implementación (Objeto).

Paso 1: Lectura del Caso de Estudio

Lee detenidamente la siguiente descripción del requerimiento para identificar las entidades principales del sistema.

“Se requiere diseñar la lógica para una aplicación de renta de scooters. En el sistema, cada **Scooter** cuenta con un número de identificación (ej. ‘S-001’), un nivel de batería (del 0 al 100) y un estado lógico que indica si está disponible para usarse o no. Los scooters pueden ejecutar tres acciones: desbloquearse para iniciar un viaje, terminar su viaje, y recargar su batería al 100%.

Por otro lado, la entidad **Usuario** se registra con su nombre y mantiene un saldo monetario en su cuenta. El usuario puede realizar dos acciones operativas: agregar más saldo a su cuenta y rentar un scooter específico.”

Paso 2: Análisis y Diccionario de Clases

Completa la siguiente estructura identificando los atributos y métodos. Recuerda que los sustantivos suelen representar Clases o Atributos, mientras que los verbos representan Métodos. Utiliza los modificadores de acceso y tipos de datos adecuados en cada línea vacía.

Entidad 1: Scooter

Atributos (Acceso privado -):

- - _____id_____ : ____String_____ (identificador)
- - __nivelBateria__ : ____Int____ (batería)
- - __estadoDisponible_ : ____boolean_____ (estado de disponibilidad)

Métodos (Acceso público +):

- + _____iniciarViaje_____() : boolean (iniciar viaje)
- + __FinalizarViaje_____() : void (finalizar viaje)
- + _____LlenarBateria_____() : void (llenar batería)

Entidad 2: Usuario

Atributos (Acceso privado -):

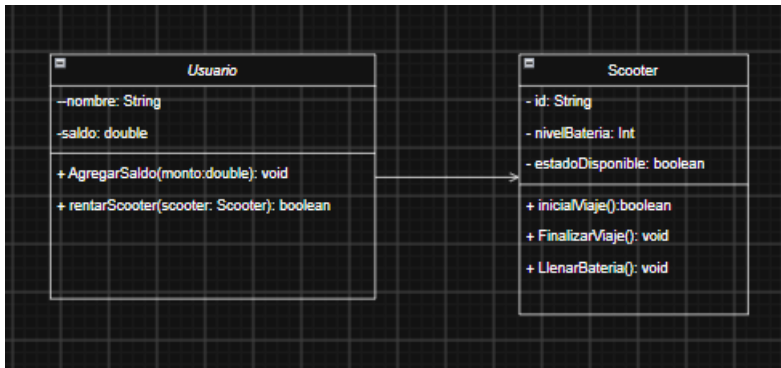
- - ____nombre____ : ____String____ (nombre)
- - ____saldo____ : ____Double____ (dinero)

Métodos (Acceso público +):

- + ____AgregarSaldo____(monto: double) : void (agregar saldo)
- + ____RentarScooter____(scooter: Scooter) : boolean (rentar scooter)

Paso 3: Diseño del Diagrama de Clases UML

A partir de la información estructurada en el paso anterior, elabora un Diagrama de Clases formal. Asegúrate de incluir los tres bloques estándar (Nombre de la clase, Atributos y Métodos) y adjuntar la captura o imagen del diagrama resultante en tu documento.



(Herramienta sugerida: Draw.io)

Paso 4: Instanciación (De la Teoría a la Realidad)

El diagrama de clases funciona como un molde estructural. A continuación, define instancias en memoria asignando valores concretos a los atributos de cada objeto para ejemplificar su estado.

Instancia de Scooter 1

- id = S-001
- nivelBateria = 100
- estadoDisponible = Si

Instancia de Usuario 1

- nombre = Emanuel
- saldo = 300

Paso 5: Pregunta de Análisis Lógico

Justifica tu respuesta a la siguiente interrogante lógica basándote en los conceptos de POO estudiados:

Si la Instancia de Usuario 1 intenta ejecutar el método rentar() enviando como parámetro un Scooter que tiene un 15% de batería, ¿cómo debería comportarse internamente la lógica del sistema?

El usuario no podría rentar, recibiría un mensaje que tiene poca batería. Solo se puede iniciar un viaje si la batería del scooter es igual o mayor que 70. Condición que estaría en el sistema. El usuario al momento de rentar(), comprobar ≥ 70 , si cumple inicia viaje, si no, no se renta